



Politechnika Krakowska
Wydział Inżynierii
Elektrycznej i Komputerowej

Kraków, 8 czerwca 2025
Grupa P1

Zastosowanie modelu LSTM do modelowania języka naturalnego

Projekt z przedmiotu Sztuczna Inteligencja

Michał Jabczyk, Kacper Świtała

Spis treści

1 Cel i opis projektu	2
2 Wstęp teoretyczny	3
2.1 Wstęp do Danych	3
2.2 Klasyczne podejścia w przetwarzaniu tekstu	3
2.3 Podejścia AI w przetwarzaniu tekstu	3
2.4 Wstęp teoretyczny do LSTM	4
2.5 Porównanie LSTM do RNN	5
3 Przygotowanie danych	6
3.1 Wybór zbioru	6
3.2 Format danych zbioru	6
3.3 Przygotowanie zbioru do uczenia	6
3.4 Lematyzacja, sposób kodowania	6
3.5 Reprezentacja danych	7
3.6 Kod do przygotowania danych - istotne fragmenty	7
4 Model LSTM przewidujący następne słowo	9
4.1 Zasada działania	9
5 Trenowanie i testowanie modelu	10
5.1 Prototyp 1	10
5.2 Prototyp 2	11
5.3 Wersja końcowa	11
6 Podsumowanie	12
7 Bibliografia	13
7.1 Pozycje internetowe	13

1 Cel i opis projektu

Głównym celem projektu było zbadanie i zaimplementowanie modelu do przewidywania następnego słowa w sekwencji tekstowej, z potencjalnym zastosowaniem w systemach autouzupełniania. Projekt skupiał się na wykorzystaniu architektury sieci neuronowej typu Long Short-Term Memory (LSTM). Dodatkowym celem było praktyczne zapoznanie się z działaniem, trenowaniem oraz ewaluacją modeli LSTM w kontekście przetwarzania języka naturalnego.

Kod wykorzystany do przetwarzania danych, trenowania modelu oraz jego testowania jest dostępny w serwisie GitHub pod adresem: <https://github.com/Sweetoos/ai-projekt-uni>

2 Wstęp teoretyczny

2.1 Wstęp do Danych

Ważne jest rozszerzenie plików danych, z którego importujemy nasze dane, ponieważ dane importowane z internetu są zazwyczaj w skompresowanej formie (np. .warc.gz, .tar.gz czy .zip). Te pliki są konwertowane do rozszerzeń bardziej przyjaznych do przetwarzania tekstu jak .jsonl czy .parquet. Dane należy oczyścić, gdyż dane mogą zawierać znaki bardziej złożone niż w alfabecie łacińskim jak np. ł, ó, ż, czy też mogą mieć różną wielkość liter i trzeba to poprawić. W importowanych zasobach kluczowe jest pozbycie się wszelkich duplikatów dla poprawy szybkości modelu językowego oraz zapewnienia różnorodności generowanego tekstu. Pomaga to zapobieganiu nadmiernego dopasowania modelu dla powtarzalnej treści. Ten proces można zaimplementować trzema podejściami: dokładnym, rozmytym oraz semantycznej deduplikacji.

2.2 Klasyczne podejścia w przetwarzaniu tekstu

Przed erą zaawansowanych modeli głębokiego nauczania, przetwarzanie tekstu było oparte o metody statystyczne i algorytmiczne. Kluczowymi podejściami, szczególnie w kontekście modelowania języka i przewidywania tekstu, należą:

N-gramy - Sekwencje N kolejnych słów (lub znaków) w tekście. Modele oparte na N-gramach przewidują następne słowo na podstawie prawdopodobieństwa jego wystąpienia po sekwencji N-1 poprzedzających słów, obliczanego na podstawie częstotliwości w korpusie treningowym. Są proste obliczeniowo, ale mają trudności z uchwyceniem zależności długoterminowych i rzadkimi słowami.

Modele Markowa - Uogólnienie N-gramów, gdzie zakłada się, że prawdopodobieństwo wystąpienia danego stanu (np. słowa) zależy tylko od ograniczonej liczby poprzednich stanów (własność Markowa).

Bag-of-Words (BoW) - Reprezentacja tekstu jako nieuporządkowanego zbioru słów, zliczająca ich wystąpienia. Choć nie służy bezpośrednio do przewidywania sekwencji, jest podstawą wielu klasycznych technik klasyfikacji tekstu.

TF-IDF (Term Frequency-Inverse Document Frequency) - Metoda ważenia słów, która ocenia ich znaczenie w dokumencie w kontekście całego korpusu. Przydatna w wyszukiwaniu informacji i kategoryzacji, ale mniej bezpośrednio w generowaniu sekwencji.

Mimo swoich ograniczeń, te metody stanowiły fundament dla rozwoju bardziej zaawansowanych technik i wciąż znajdują zastosowanie w jakichś prostszych zadaniach.

2.3 Podejścia AI w przetwarzaniu tekstu

Dokładne - skupia się na zidentyfikowaniu i usunięciu kompletnie identycznych dokumentów. To podejście generuje klucz dla każdego dokumentu oraz grupuje te dokumenty przez ich klucze do kubełków, tak by trzymać jeden dokument na kubek. Zaletą takiego podejścia jest efektywność, szybkość oraz niezawodność, a wadą jest ograniczenie do wykrywania idealnego dopasowania do treści, co może spowodować ominięcie semantycznie porównywalnych dokumentów z drobnymi wariacjami.

Rozmyte - adresuje prawie zduplikowane treści przy użyciu sygnatur MinHash i Locality-Sensitive Hashing (LSH). Proces wpierv wylicza klucze MinHash dla dokumentów, po czym używa LSH do grupowania podobnych dokumentów do kubelków. 1 dokument może należeć do więcej niż jednego kubelka. Następnie trzeba wyliczyć podobieństwo Jaccarda, czyli takie, które porównuje podobieństwo między dokumentami w tych samych kubelkach, porównując stopień wspólności tych elementów na przykład zbioru słów względem wszystkich unikalnych elementów w obu dokumentach. Bazując na tym podobieństwie przekształcamy macierz podobieństwa do grafu i identyfikujemy połączone komponenty w grafie. Dokumenty w połączonym komponencie są rozpatrywane jako rozmyte duplikaty, a następnie usuwane z datasetu.

Semantyczne - reprezentuje najbardziej zaawansowane podejście wykorzystujące nowoczesne modele osadzania (embedding), które uchwytują znaczenie semantyczne danych, w połączeniu z technikami klasteryzacji do grupowania semantycznie podobnych treści. Badania wykazały, że deduplikacja semantyczna skutecznie zmniejsza rozmiar zbioru danych, jednocześnie utrzymując lub nawet poprawiając wydajność modelu. Jest szczególnie przydatna w wykrywaniu parafraz, tłumaczeń tego samego materiału oraz treści o identycznym znaczeniu. Aby dokonać deduplikacji semantycznej wpierv trzeba przekształcić każdy punkt danych na wektor za pomocą wstępnie wytrenowanego modelu. Grupujemy te wektory w k klastrów przy użyciu algorytmu k-średnich (k-means). Wewnątrz każdego takiego klastra obliczane są pary podobieństw cosinusowych. Każdej parze danych, której podobieństwo cosinusowe przekracza ustalony próg, przypisuje się status semantycznych duplikatów. Z każdej grupy semantycznych duplikatów w klastrze zachowuje się tylko jeden reprezentatywny punkt danych, reszta jest usuwana.

2.4 Wstęp teoretyczny do LSTM

Sieci LSTM są specjalnym rodzajem RNN (Recurrent Neural Network), które zostały opracowane, aby znacznie lepiej radzić sobie z zapamiętywaniem przez długi czas. W standardowych sieciach problemem jest tendencja do „zapominania” informacji z wcześniejszych etapów sekwencji, gdy sekwencja stanie się długa. LSTM został stworzony po to, by poradzić sobie z tym problemem. Mają wbudowaną taką wewnętrzną „pamięć” zwaną stanem komórki (cell state) oraz specjalnych struktur kontrolujących przepływ informacji, zwykle zwanych bramkami.

Stan komórki jest główną linią pamięci, biegnącą przez całą sieć LSTM od jednego kroku przetwarzania sekwencji do następnego. Informacje na tej linii mogą być przechowywane, modyfikowane lub usuwane w kontrolowany sposób. Mogą również przebiegać w dużej mierze niezmiennie, co pozwala sieci „pamiętać” istotne rzeczy z odległej przeszłości sekwencji. Jest to istotna różnica pomiędzy LSTM a prostszymi RNN.

Struktury kontrolujące (bramki) można podzielić na 3 typy:

1. Mechanizm Zapominania (Forget Gate)
2. Mechanizm Wejściowy (Input Gate)
3. Mechanizm Wyjściowy (Output Gate)

Te struktury działają w powyższej kolejności. W bramce zapominania jest podejmowana decyzja, które informacje z poprzedniego stanu komórki powinny zostać zapomniane lub odrzucone. Następnie w bramce wejściowej LSTM decyduje, jakie nowe informacje z bieżącego fragmentu danych (np. aktualizowanie danego słowa) są na tyle ważne, by je

zapisać w stanie komórki. Ten mechanizm składa się z dwóch części - wpierw identyfikuje, które wartości z nowych danych warto zaktualizować, a potem tworzy listę potencjalnych nowych informacji, które mogłyby zostać dodane. Łącząc te kroki pozwala nam selektywnie zaktualizować stan komórki o nowe istniejące dane. Na końcu za pomocą bramki wyjściowej na podstawie zaktualizowanego stanu komórki (która jest mieszanką starych i nowych informacji), LSTM decyduje, co powinno być wynikiem przetwarzania w bieżącym kroku. Ten wynik staje się również tzw. stanem ukrytym, który jest formą krótkoterminowej pamięci przekazywanej do następnego kroku przetwarzania i używanej przez mechanizmy kontrolne w kolejnym etapie. Mechanizm wyjściowy filtruje informacje ze stanu komórki, aby wygenerować użyteczne wyjście.

2.5 Porównanie LSTM do RNN

LSTM ma przewagę nad tradycyjnymi RNN zdolnością do radzenia sobie z problemem zanikającego gradientu oraz w mniejszym stopniu z problemem eksplodującego gradientu. W standardowych RNN'ach, podczas procesu uczenia metodą propagacji wstecznej w czasie, gradienty są mnożone przez te same macierze wag na każdym kroku czasowym. Zanikający gradient jest w przypadku małych liczb, natomiast eksplodujący gradient w przypadku dużych. W przypadku zanikającego gradientu prostego problemem jest wolne uczenie lub nawet jego zatrzymanie oraz niemożność nauczenia się długoterminowych zależności, czyli jak gradient zanika, sieć nie jest w stanie „dowiedzieć się” jak błąd na końcu długiej sekwencji zależy od tego, co działo się na jej początku. W przypadku eksplodującego gradientu gradient uczy się niestabilnie, może przeskoczyć jakieś iteracje uczenia. LSTM zapobiega temu w taki sposób, że stany komórki są głównie addytywne, a nie multiplikatywne przez kolejne macierze wag, co pozwala gradientom przepływać przez długie sekwencje w bardziej niezminionej formie. W przypadku bramek to w bramce zapominania gradient może „nauczyć się” resetować pamięć, co może pomóc w przerywaniu długich łańcuchów mnożeń, które prowadzą do zanikania. Z kolei bramka wejściowa kontroluje dodawane informacje, które wpływają do gradientu.

3 Przygotowanie danych

3.1 Wybór zbioru

Do projektu wykorzystujemy dataset [chirunder/text_messages z Huggingface](https://huggingface.co/datasets/chirunder/text_messages). Zbiór zawiera interesujące nas dane - wiele krótkich zdań/wiadomości, napisanych prostym językiem angielskim.

3.2 Format danych zbioru

Zbiór danych składa się z dwóch plików parquet, stworzonych z jednej kolumny text. Zbiór zawiera 11.6 miliona zdań, każdy jako osobny wiersz o długości od 2 do 3010 znaków. Zdania są w języku angielskim. Zaczynają się wielką literą, a kończą kropką. Zdania są wiadomościami tekstowymi pochodzącymi z komunikacji pomiędzy ludźmi. Do trenowania wykorzystujemy jeden z plików, o wadze 281.7 MB.

3.3 Przygotowanie zbioru do uczenia

Aby model mógł efektywnie się uczyć i sugerować poprawne odpowiedzi, surowe dane należy przetworzyć. W tym celu usuwamy interpunkcję, liczby, zamieniamy słowa na małe litery. Czyścimy zbiór z niechcianych odpowiedzi. Zbiór przetwarzamy za pomocą skryptu napisanego w języku python. Jest on zapisany w pliku src/1_prepare_data.py.

Dane przygotowujemy w następujący sposób:

1. Losowa część rekordów jest usuwana (w przypadku, gdy potrzebujemy mniejszy dataset do testów)
2. Tekst jest zamieniany na wyłącznie małe litery (lowercase)
3. Usuwane są znaki interpunkcyjne oraz liczby
4. Tekst jest „trymowany”, usuwane są początkowe i końcowe spacje
5. Tekst jest dzielony na słowa w słowniku
6. Dokonujemy lematyzacji
8. Ze zdań tworzone są dane do trenowania - ciąg trzech słów mapowany jest do następnego wyrazu w zdaniu. Dane są zapisywane jako liczby, korzystając ze słownika.
9. Przypadki są zapisywane do pliku w formacie parquet, który umożliwia ładowanie danych do pamięci operacyjnej w częściach.
10. Metadane, czyli informacje o słowniku oraz lookup table słownika są zapisywane w formacie pickle, gdyż mogą być załadowane w całości ze względu na mały rozmiar.

3.4 Lematyzacja, sposób kodowania

Nie chcemy, by model zajmował się rzadkimi słowami, gdyż zwiększa to złożoność modelu dając minimalne korzyści. Dlatego wybierane jest 10 000 najpopularniejszych słów, które chcemy podpowiadać. Pozostałe są zamieniane na token <UNK> Tworzony jest słownik, tablica z 10 001 elementami, w której każde słowo występuje tylko raz. Dzięki temu można przypisać każdemu słowu liczbę będącą indeksem tego słowa w tablicy.

3.5 Reprezentacja danych

Dane dzielimy na kilka plików:

- Plik słownika (vocab.pkl)
- Plik z danymi treningowymi

Plik słownika zawiera prostą strukturę z trzema danymi:

- Lookup table word to index - słownik mapujący słowo do liczby
- Lookup table index to word - słownik mapujący liczbę do słowa
- vocab_size - liczbę słów w słowniku

Dane są zapisywane w formacie pickle (pkl) za pomocą biblioteki słownika.

Plik z danymi treningowymi jest zapisywany w formacie parquet, który umożliwia ładowanie danych do pamięci operacyjnej w kawałkach. Format danych jest następujący:

- Zbiór składa się z listy słowników
- Każdy słownik zawiera 2 pola, x i y
- Pole x składa się z krotki (tuple) zawierającej 1-3 wyrazy poprzedzające następne słowo, jako liczba
- Pole y jest liczbą reprezentującą słowo następujące po ciągu

Przykładowo, dla słownika ale = 0, ala = 1, ma = 2, kota = 3 i jedynej danej treningowej ale ala ma -> kota, plik będzie zawierał następującą strukturę:

```
[
  { "x": (0, 1, 2), "y": 3 }
]
```

3.6 Kod do przygotowania danych - istotne fragmenty

```
def clean_text(text):
    text = text.lower()
    text = re.sub(rf"[{re.escape(string.punctuation)}]", "", text) # remove
punctuation
    text = re.sub(r"\d+", "", text) # TODO: instead of removing just numbers, remove
the entry entirely
    text = re.sub(r"\s+", " ", text).strip() # clean up extra spaces
    return text
```

```
VOCAB_LIMIT = 10000
UNK_TOKEN = "<UNK>"
```

```
cleaned_sentences = []
for sentence in sentences:
    cleaned = clean_text(sentence)
    if len(cleaned.split()) >= 3:
        if random.random() > DROP_RATE:
            cleaned_sentences.append(cleaned)

print(f"Cleaned sentences: {len(cleaned_sentences)}")
```

```
words = " ".join(cleaned_sentences).split()
word_counts = Counter(words)
```

```
most_common_words = word_counts.most_common(VOCAB_LIMIT)
vocab = [word for word, _ in most_common_words]
```

```

vocab.append(UNK_TOKEN)

word2idx = {w: i for i, w in enumerate(vocab)}
idx2word = {i: w for w, i in word2idx.items()}
vocab_size = len(vocab)
print(f"Vocabulary size: {vocab_size}")

# Dataset creation
def make_dataset(sentences, word2idx, context_size=3):
    data = []
    i = 0
    for sentence in sentences:
        i += 1
        if i % 50000 == 0:
            print(f"Processing sentence {i}/{len(sentences)}")

        tokens = sentence.split()
        if len(tokens) < context_size + 1:
            continue
        for i in range(len(tokens) - context_size):
            context = tokens[i:i+context_size]
            target = tokens[i+context_size]
            try:
                context_idx = [word2idx.get(w, word2idx[UNK_TOKEN]) for w in context]
                target_idx = word2idx[target]
                data.append({ "x": context_idx, "y": target_idx })
            except KeyError:
                continue # skip if any word is not in vocab
    return data

dataset_meta = {
    "word2idx": word2idx,
    "idx2word": idx2word,
    "vocab_size": vocab_size
}

```


4 Model LSTM przewidujący następne słowo

W ramach projektu model LSTM został zaimplementowany za pomocą frameworka PyTorch.

4.1 Zasada działania

```
import torch
import torch.nn as nn
import torch.optim as optim

embedding_dim = 50
hidden_dim = 100

class LSTMWordPredictor(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim):
        print(f"Initializing model with vocab_size={vocab_size},
embedding_dim={embedding_dim}, hidden_dim={hidden_dim}")
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x):
        emb = self.embedding(x)
        out, _ = self.lstm(emb)
        out = self.fc(out[:, -1, :]) # Use output from last timestep
        return out
```

Mamy klasę `LSTMWordPredictor` z konstruktorem modelu (`__init__`) i parametrami `vocab_size`, `embedding_dim`, `hidden_dim`. `vocab_size` oznacza liczbę całkowitą unikalnych słów w słowniku, na podstawie którego model będzie operował, `embedding_dim` jest wymiarem wektorów osadzeń (embeddings), gdzie każde słowo będzie reprezentowane przez gęsty wektor o tej długości. `hidden_dim` jest liczbą jednostek w warstwie ukrytej LSTM, definiuje „pojemność” pamięci modelu. W konstruktorze definiujemy 3 warstwy. Pierwszą z nich, czyli warstwę osadzenia (`self.embedding`) odpowiada za transformację indeksów słów (będących liczbami całkowitymi) na gęste wektory liczbowe, czyli osadzenia. Jest to fundamentalny krok, pozwalający modelowi na naukę semantycznych reprezentacji poszczególnych słów. Kolejną inicjalizowaną warstwą jest główna warstwa LSTM (`self.lstm`), która przetwarza sekwencje wektorów osadzeń (embeddingów) dostarczonych przez poprzednią warstwę. Argument `batch_first` informuje warstwę, że dane wejściowe będą miały wymiarowość, gdzie pierwszy wymiar to rozmiar batcha. Ostatnią warstwą jest warstwa w pełni połączona (liniowa) (`self.fc`), która ma za zadanie zmapowanie wyjścia z warstwy LSTM (które ma `hidden_dim` wymiarów) na wektor o rozmiarze `vocab_size`. Ten finalny wektor reprezentuje logity, czyli nieznormalizowane prawdopodobieństwa, dla każdego słowa w słowniku, wskazując, które z nich jest najbardziej prawdopodobne jako następne w sekwencji.

Metoda `forward` definiuje sposób, w jaki dane przepływają przez model podczas predykcji (tzw. forward pass). Na wejściu (`x`) otrzymuje tensor zawierający sekwencje indeksów słów. Najpierw dane te są przekazywane przez warstwę LSTM. Warstwa LSTM zwraca pełne wyjście dla każdego kroku czasowego oraz ostatni stan ukryty i stan komórki (które w tym konkretnym przypadku nie są bezpośrednio wykorzystywane do dalszej predykcji, ale są kluczowe dla wewnętrznego działania LSTM). Do przewidzenia następnego słowa wykorzystujemy jedynie wyjście LSTM z ostatniego kroku czasowego analizowanej sekwencji

wejściowej. To wyjście jest następnie przekazywane przez warstwę w pełni połączoną, aby uzyskać wspomniany wcześniej rozkład prawdopodobieństwa nad całym słownikiem. Ostatecznie model zwraca te logity jako wynik swojej predykcji.

5 Trenowanie i testowanie modelu

5.1 Prototyp 1

Prototyp 1 ma poniższe parametry:

- 10% danych do treningu
- 5, a następnie 10 epok
- Wszystkie słowa, bez filtra najpopularniejszych
- `learning_rate = 0.01`, `embedding_dim = 50`, `hidden_dim = 100`
- Rozmiar: 65.8 MB (plik pth) + 2.5 MB (vocab.pkl)

Trenowanie jednej epoki trwało około 5 minut. Wyniki testów dla 5 epok są następujące:

```
Initializing model with vocab_size=108514, embedding_dim=50, hidden_dim=100
Input: we are going to --> be
Input: we are --> we are not going to be a good idea to be a
Input: the iphone --> the iphone is a little more than
Input: the --> the first time i have a problem with the new one
Input: i believe --> i believe that the car is a little more than a few
Input: not --> not sure if i can get a new thread to the
Input: i --> i will be able to get a good deal with the
Input: what were --> what were going to be the same thing that i have been
```

Natomiast po 10 epokach model zachowuje się następująco:

```
Initializing model with vocab_size=108514, embedding_dim=50, hidden_dim=100
Input: we are going to --> be
Input: we are --> we are not going to be a lot of time to get
Input: the iphone --> the iphone is a little more expensive and i am not sure
Input: the --> the best way to do it with the other side of
Input: i believe --> i believe that the guy who is a good thing to do
Input: not --> not lobbing the forum and i was thinking about the same
Input: i --> i think it will be a good looking car and i
Input: what were --> what were talking about the same thing i have to do it
```

Model poprawnie przewiduje wyrazy, które mogą następować po sobie. Utworzone ciągi nie tworzą jednak logicznych zdań. Aby poprawić model, w następnej iteracji wykorzystamy pełny zbiór danych. Obecny model generuje wyłącznie krótkie, popularne słowa. Jest to w naszym przypadku dużą zaletą. Aby model nie uczył się rzadkich wyrazów, które nie będą chętnie wybierane przez użytkowników, postanawiamy trenować zbiór 10 000 najpopularniejszymi wyrazami, a resztę zastępować tokenem `<UNK>`. Ta zmiana powinna także znacząco zmniejszyć rozmiar modelu.

5.2 Prototyp 2

Prototyp 2 ma poniższe parametry:

- 100% danych do treningu
- 1 epoka
- 10 tysięcy najpopularniejszych słów + token <UNK>
- `learning_rate = 0.01`, `embedding_dim = 50`, `hidden_dim = 100`
- Rozmiar: 6.3 MB (plik pth) + 201 KB (vocab.pkl)

Trenowanie jednej epoki trwało około 43 minuty. Wyniki testów są następujące:

```
Input: we are going to --> be
Input: we are --> we are not a fan of the car and i have a
Input: the iphone --> the iphone and i have a question about the car and i
Input: the --> the same thing is the same thing is that the same
Input: i believe --> i believe that the car is a good thing to do with
Input: not --> not a problem with the car and i have a few
Input: i --> i have a few questions and i have been looking for
Input: what were --> what were you looking for a good idea to do it and
```

Generowane ciągi słów mają gramatyczny sens, ale model szybko „gubi się” i ztraca sens wypowiedzi. Zdarzają się pętle (np. `the same thing is the same thing is that the same`). Dużym problemem zdaje się mały kontekst danych, będący jedynie trzema ostatnimi słowami. Zdaje się on powodować zdania bez sensu (np. `what were you looking for a good idea to do it and`). Zwiększenie kontekstu do 6 czy 10 wyrazów znacznie zwiększy złożoność i czas uczenia się, dlatego decydujemy się sprawdzić większą ilość epok przy niezmienionej długości kontekstu. Mamy nadzieję, że uda się wygenerować satysfakcjonujące wyniki bez zwiększenia kontekstu.

5.3 Wersja końcowa

Wersja końcowa ma poniższe parametry:

- 100% danych do treningu
- 5 epok
- 10 tysięcy najpopularniejszych słów + token <UNK>
- `learning_rate = 0.01`, `embedding_dim = 50`, `hidden_dim = 100`
- Rozmiar: 6.3 MB (plik pth) + 201 KB (vocab.pkl)

Trenowanie zajęło 5x 43 min = 3h 35 min. Wyniki testów są następujące:

```
Initializing model with vocab_size=10001, embedding_dim=50, hidden_dim=100
Input: we are going to --> be
Input: we are --> we are going to be a good one for the first time
Input: the iphone --> the iphone is a great idea of how much of a lot
Input: the --> the same thing as the other one is a good idea
Input: i believe --> i believe it is a good idea to me and i have
Input: not --> not a problem with the stock rom and the phone is
Input: i --> i have a few more pics of the new ones and
Input: what were --> what were you doing with the new one and the other one
```

Efekty są już zadowalające, przewidywany tekst ma poprawną składnię, a zachowywanie sensu wypowiedzi jest porównywalne do prostych modeli autouzupełniania dostępnych w nowoczesnych urządzeniach mobilnych. Nadal widoczne jest szybkie zapominanie, ale model działa znacznie lepiej niż w prototypie 2. Kolejnym widocznym problemem jest nadużywanie

przedimków (the, a), spójników (and), przyimków, czy krótkich słów (is, it, for). Są to zwroty często używane w krótkich wiadomościach tekstowych. Model zdaje się preferować te słowa, gdyż występują nader często w danych testowych. Gdybyśmy chcieli generować długie teksty, byłby to duży problem, który należałoby mitygować poprzez odpowiednie dobranie danych i zmniejszenie częstotliwości występowania tych wyrazów. Jako że celem projektu jest podpowiadanie słów wiadomości tekstowych, problem ten jest w rzeczywistości zaletą - użytkownicy często wybierają krótkie wiadomości zawierające wiele tego typu słów. Z tego powodu uznajemy model za dostatecznie dobry dla naszego przypadku.

6 Podsumowanie

Projekt miał na celu zbudowanie i przetestowanie modelu opartego na architekturze LSTM do przewidywania następnego słowa w sekwencji, z myślą o zastosowaniach takich jak autouzupełnianie tekstu. Realizacja składała się z kilku kroków takich jak oczyszczenie danych tekstowych (w tym lematyzację i filtrowanie słownika), implementację modelu LSTM przy użyciu biblioteki PyTorch oraz iteracyjne trenowanie i testowanie różnych konfiguracji modelu.

Eksperymenty wykazały kilka wniosków:

- Projekt dostarczył pewnego doświadczenia jak działa sieć LSTM, od przygotowania danych, przez implementację modelu po jego trenowanie i ewaluację.
- Jakość i sposób przygotowania danych wejściowych ma fundamentalne znaczenie dla wydajności modelu. Procesy takie jak lematyzacja, czyszczenie tekstu, filtrowanie słownika (np. do najpopularniejszych słów i użycie tokenu <UNK>) oraz odpowiednie kodowanie danych są niezbędne, aby model mógł efektywnie się uczyć i generować sensowne wyniki.
- Ilość danych treningowych oraz odpowiednia liczba epok treningu są istotne. Większa ilość danych pozwala modelowi lepiej generalizować i uczyć się bardziej złożonych wzorców, kiedy odpowiednia liczba epok zapewnia, że model ma wystarczająco dużo czasu na konwergencję, choć należy uważać na przetrenowanie.

Mimo pewnych ograniczeń udało się opracować model LSTM, który jest stosunkowo niewielki pod względem zajmowanej pamięci, a jednocześnie generuje predykcje na zadowalającym poziomie, szczególnie w autouzupełnianiu małych wartości tekstowych. Model poprawnie uczy się podstawowych struktur gramatycznych i częstych sekwencji słów.

7 Bibliografia

7.1 Pozycje internetowe

- Malik Saad Ahmed, Text Generation Using LSTMs and GPT-2 In PyTorch <https://medium.com/@MalikSaadAhmed/text-generation-using-lstms-and-gpt-2-in-pytorch-8097c948ccd8> Dostęp 03.04.2025
- Amit Beiweiss & Nicole Luo, Mastering LLM Techniques Data Preprocessing <https://developer.nvidia.com/blog/mastering-llm-techniques-data-preprocessing> Dostęp 17.05.2025
- Christopher Olah, Understanding LSTMs <https://colah.github.io/posts/2015-08-Understanding-LSTMs/> Dostęp 18.05.2025
- Czym jest komórka LSTM i dlaczego jest wykorzystywana w implementacji RNN? <https://pl.eitca.org/sztuczna-inteligencja/eitc-ai-dlrf-g%C5%82%C4%99bokie-uczenie-z-tensorflow/rekurencyjne-sieci-neuronowe-w-tensorflow/Przyk%C5%82ad-rnn-w-tensorflow/egzamin-prze%C4%85d-rnn-przyk%C5%82ad-w-tensorflow/co-to-jest-kom%C3%B3rka-lstm-i-dlaczego-jest-u%C5%BCywana-w-implementacji-rnn/> Dostęp 02.06.2025
- Daniel Jurafsky & James H. Martin, Speech and Language Processing ch. 3, <https://web.stanford.edu/~jurafsky/slp3/3.pdf> Dostęp 02.06.2025